

嵌入式各章总结II

第五章

备考嵌入式操作系统的调度算法部分确实需要理清不少逻辑，特别是各种算法的判定条件和特殊情况！这份由桑楠教授编写的《嵌入式操作系统》课件结构非常清晰。

根据文件内容，我为你整理了这部分最核心的考点，主要分为**基础概念**、**三大经典调度算法**以及**并发机制**（优先级反转与死锁）三个主要模块：

一、调度基础概念

这部分通常会以选择题、填空题或简答题的形式出现，考察对基本术语的理解。

- **任务特性的四个维度**：动态性（状态不断变化）、并发性、异步独立性（不存在前驱与后继关系）、同步性（相互依赖）。
- **时间约束的三大指标**：截止时间（必须完成的时间）、执行周期（重复出现并处理完成的时间间隔）、延迟时间（允许等待的时间）。
- **机制与策略的区别**：
 - 调度策略：确定哪个任务先执行、执行时间等的规则，本质是为任务确定优先级。
 - 调度机制：根据调度策略来安排任务的具体执行（如创建、就绪、挂起等）。
 - 两者共同完成内核的调度功能。
- **调度分类**：需要分清静态调度（运行前确定）与动态调度（运行时确定）的区别。需要绝对可预测性的系统一般不使用动态调度。

二、三大经典实时调度算法（必考重点）

这部分是核心计算题和综合分析题的高频考区。你需要熟记每种算法的**调度规则**和**可调度性判定公式**。

1. RMS (速率单调调度算法 - Rate Monotonic Scheduling)

- **类型**：最好的静态优先级调度算法。
- **核心规则**：任务的周期越小，优先级越高，即优先级由周期唯一决定（ $Pri_i = \frac{1}{P_i}$ ）。

- **假设前提：**所有任务是周期的、独立的、可在任何位置被抢占，且相对截止时间等于周期 ($D_{\{i\}}=P_{\{i\}}$) 。
- **可调度性判定（充分条件）：**系统可调度的充分条件是 CPU 的利用率 $\leq n(2^{1/n}-1)$ ，其中 n 为任务个数 。
- **局限性：**可调度的 CPU 使用率上限较低（当 n 趋于无穷时，上限约为 $\ln 2$ 或 0.693），对大多数系统来说不可接受 。

2. EDF (最早截止时间优先算法 - Earliest Deadline First)

- **类型：**最好的单处理器动态调度算法 。
- **核心规则：**截止时间越短的任务具有越高的优先权 。
- **可调度性判定（充要条件）：**任务集可调度的充要条件是 CPU 的利用率 ≤ 1 。
- **优缺点：**优点是充分利用 CPU 的计算能力（上限 100%）；缺点是调度开销大，且系统临时过载时，不能确定哪个任务的截止时间得不到满足 。

3. LLF (最小松弛度优先算法 - Least-Laxity-First)

- **类型：**单处理器动态调度算法 。
- **核心规则：**任务的松弛度（空闲时间）越短，任务的优先级越高 。
- **核心公式：**任务的空闲时间 = 截止时间 - 任务剩余执行时间 。
- **可调度性判定（充要条件）：**与 EDF 相同，CPU 的利用率 ≤ 1 。

三、 优先级反转及其解决方案（高频简答/分析题）

这部分极其重要，经常要求描述现象并对比两种解决方案。

- **优先级反转现象：**高优先级任务需要等待低优先级任务释放资源，而低优先级任务又正在等待（或被其抢占）中等优先级任务执行的现象。最终导致最高优先级任务最后执行完成 。
- **解决方案一：优先级继承协议**
 - **机制：**当任务阻塞了更高优先级任务时，它继承被阻塞任务的最高优先级来执行临界区，退出时恢复原优先级 。
 - **缺点：**可能出现阻塞链，加长阻塞时间，甚至不能保证避免死锁 。
- **解决方案二：优先级天花板协议**

- **机制**：系统为每个资源分配一个“天花板”优先级（该资源可能被请求的最高优先级）。当任务占用资源且具有比天花板更高的优先级任务请求时，将其优先级提升到天花板等级。
- **优点（与继承协议的对比）**：可以避免死锁问题和阻塞链问题；执行效率更高（天花板只需改变一次优先级，而继承可能多次改变）。

四、死锁 (Deadlock)

- **产生死锁的四个必要条件**：资源独占、不可抢占、请求与保持、循环等待。
- **预防与解除**：预防的核心是破坏四个条件之一（如使用银行家算法），解除则是消除四个条件之一。

你希望我用文件中的具体数据表（例如 $T_{\{1\}}$, $T_{\{2\}}$, $T_{\{3\}}$ 的执行时间和周期）为你演示一下 RMS 或 EDF 的可调度性计算过程吗？

第六章

备战期末的开卷考试，这份关于“内核移植”的课件非常有针对性。特别是里面大量结合了 aCoral 和 ARM9（S3C2410/2440）的具体底层代码，和你平时的实践项目完全契合。

我为你提炼了可以在考场上快速查阅的核心考点结构，建议考试时重点锁定以下三大模块：

一、内核移植核心概念与流程

这部分主要应对概念简答题，理清“是什么”和“为什么”。

- 内核移植是指将内核代码从一种硬件平台（CPU或SOC芯片）转移到另一种硬件平台上运行。
- 移植的驱动原因包括基本内核过于庞大、厂家仅提供Demo、以及必须结合工程师设计的具体硬件电路进行适配（硬件可裁剪性）。
- 移植的通用过程分为两大核心部分：硬件抽象层（HAL）移植和项目移植。
- 在基于参考板的移植中，通常需要先编译测试参考板内核，然后添加开发板初始项并移植 Flash、SDRAM、以太网口等特定驱动程序。

二、HAL层移植（考点密集区）

这是嵌入式考试中极易出现代码分析或填空题的区域，务必关注课件中出现的 ARM 汇编指令。

- HAL 移植必须改写与硬件高度相关的代码，主要涵盖五大类：启动过程、中断操作、任务切换、内存管理和驱动程序。
- **启动接口**：涉及修改链接文件（如 `aCoral.lds`）来确定代码段入口和内存起始地址，例如设定 RAM 起始于 `0x30000000`。
- **中断接口**：在 ARM 体系中，中断入口需要使用 `stmfd sp!, {r0-r12,lr}` 指令来保护通用寄存器和程序计数器。
- **中断嵌套支持**：为了支持中断嵌套，必须将当前程序状态寄存器（`spsr`）压栈保护，并通过操作 `cpsr` 切换到 `SVCMODE|NOIRQ` 模式。
- **任务切换接口**：`HAL_CONTEXT_SWITCH` 函数的重点是上下文环境的保存与恢复，需要将旧任务的寄存器状态压栈，保存旧上下文栈指针，并提取新任务的上下文栈指针来恢复执行环境。
- **时钟接口**：Ticks 的移植依赖于具体芯片的定时器配置，课件明确展示了在初始化定时器控制寄存器（`rTCON`）时，S3C2410 与 S3C2440 之间的细微取值差异。
- **代码规范**：为了保证可维护性，硬件相关的接口应遵循规范命名，aCoral 中统一采用 `HAL_XXX` 的格式。

三、项目移植与工程实施

这部分偏向于工程实践，常考查对整个开发环境和烧写流程的理解。

- 项目移植主要解决不同开发环境间编译器、汇编器、链接器规则不一致，以及 C 代码兼容性的问题。
- aCoral 的移植环境在物理文件树上主要分为 HAL 环境（包含板级配置、头文件等）和 Drivers 环境（包含外设驱动代码）。
- 项目移植的实际操作步骤包括：创建项目目录环境、拷贝芯片相关文件、拷贝并变更驱动文件，以及调整内核配置。
- 目标代码生成后，通常使用 minicom 连接串口进行调试，并借助 DNW 工具将内核烧写到目标开发板上运行。

针对课件第29页的思考题速查：

- 思考题1和2的答案完全可以从上述“概念与流程”及“HAL层移植”的分类中直接提取。
- 思考题3关于 S3C2410 复位异常的处理，直接对应课件的“统一入口处理”，软件上需要使用 `b ResetHandler` 等分支指令将程序强制跳转到相应的异常处理函数入口。

需要我针对这里面的某段 ARM 汇编代码（比如任务切换时的上下文栈指针操作）再做具体的寻址方式或执行逻辑拆解吗？